

INSTITUTO TECNOLÓGICO DE LAS AMÉRICAS (ITLA)

Carrera: Tecnólogo en Desarrollo de Software

Materia: Programación 2 (Programación Orientada a Objetos)

PRÁCTICA 3

Estructuras de Datos (Listas, Pilas y Colas) en C#

Docente: Eduandy Isabel Cruz Abreu

Estudiante: Franklyn Enmanuel Santana Rodríguez

Matrícula: 2025-2089

Modalidad: Proyecto Individual

Repositorio en GitHub:

https://github.com/Osiris3939/Practica3_Prog2

Santo Domingo, República Dominicana

Introducción

Las **estructuras de datos lineales** son uno de los componentes fundamentales del desarrollo de software, ya que determinan cómo se organiza, accede y manipula la información en memoria durante la ejecución de un programa. En C#, el espacio de nombres *System.Collections.Generic* provee implementaciones robustas y fuertemente tipadas de estas estructuras, permitiendo elegir la que mejor se ajusta al comportamiento que exige cada problema del mundo real.

La **Lista (List<T>)** es la estructura más flexible de las tres: almacena sus elementos en un arreglo dinámico internamente redimensionable, lo que permite **acceso indexado** en tiempo constante (O(1)) a cualquier posición, así como búsquedas, inserciones y eliminaciones dinámicas en cualquier punto de la colección, no solo en los extremos.

La **Cola (Queue<T>)** implementa el principio **FIFO — First In, First Out** (el primero en entrar es el primero en salir), modelando con precisión cualquier proceso de atención por orden de llegada, como una fila de clientes esperando ser atendidos: quien llega primero, se atiende primero.

La **Pila (Stack<T>)**, en cambio, implementa el principio **LIFO — Last In, First Out** (el último en entrar es el primero en salir), lo cual resulta ideal para el control de tareas pendientes, historiales de navegación, o cualquier flujo donde lo más reciente debe resolverse primero antes de continuar con lo anterior.

La presente práctica desarrolla un sistema de consola modular en C# que integra las tres estructuras en módulos independientes, cada uno con sus propias validaciones de negocio, demostrando de forma práctica cuándo y por qué conviene elegir cada una.

Arquitectura del Sistema y Módulos Desarrollados

Menú Principal y Submenús

El sistema se organiza mediante una estructura modular basada en un ciclo while combinado con una instrucción switch, que mantiene al programa en ejecución hasta que el usuario decide salir, permitiendo navegar libremente entre los tres módulos (Estudiantes, Clientes y Tareas) sin perder en ningún momento los datos cargados en memoria durante la sesión.

```
bool continuarEjecucion = true;
while (continuarEjecucion)
{
    MostrarMenuPrincipal();
    string opcion = Console.ReadLine();
    switch (opcion)
    {
        case "1": SubMenuEstudiantes(); break;
        case "2": SubMenuColaClientes(); break;
        case "3": SubMenuPilaTareas(); break;
        case "4": EjecutarDemostracionCompleta(); break;
        case "0": continuarEjecucion = false; break;
        default: Console.WriteLine("Opción no válida."); break;
    }
}
```

Las listas, colas y pilas se declaran como variables a nivel de clase (fuera del ciclo while), de modo que conservan su estado —los estudiantes agregados, los clientes en espera, las tareas pendientes— durante toda la ejecución del programa, independientemente de cuántas veces el usuario entre y salga de cada submenú.

Módulo 1: Lista de Estudiantes (List<Estudiante>)

Se definió la clase *Estudiante* con las propiedades *Nombre* y *Calificacion*, y una *List<Estudiante>* que permite agregar, eliminar por nombre y listar todos los registros, con acceso indexado directo a cualquier posición de la colección.

```
public class Estudiante
{
    public string Nombre { get; set; }
    public double Calificacion { get; set; }
}

List<Estudiante> listaEstudiantes = new List<Estudiante>();

public void AgregarEstudiante(string nombre, double calificacion)
{
    if (listaEstudiantes.Any(e => e.Nombre.Equals(nombre,
StringComparison.OrdinalIgnoreCase)))
    {
        Console.WriteLine($"[Error]: Ya existe un estudiante registrado con el nombre
'{nombre}'.");
        return;
    }
    if (calificacion < 0 || calificacion > 100)
    {
        Console.WriteLine("[Error]: La calificación debe estar en un rango de 0 a
100.");
        return;
    }
    listaEstudiantes.Add(new Estudiante { Nombre = nombre, Calificacion = calificacion
});
    Console.WriteLine($"[Éxito]: Estudiante '{nombre}' agregado correctamente.");
}

public void EliminarEstudiantePorNombre(string nombre)
{
    Estudiante encontrado = listaEstudiantes.FirstOrDefault(e =>
        e.Nombre.Equals(nombre, StringComparison.OrdinalIgnoreCase));
    if (encontrado == null)
    {
        Console.WriteLine($"[Error]: No se encontró ningún estudiante llamado
'{nombre}'.");
        return;
    }
    listaEstudiantes.Remove(encontrado);
    Console.WriteLine($"[Éxito]: Estudiante '{nombre}' eliminado correctamente.");
}
```

Antes de agregar un nuevo estudiante, el método valida que no exista ya un registro con el mismo nombre (evitando duplicados) y que la calificación ingresada se encuentre dentro del rango válido de 0 a 100. Para la eliminación, se busca primero el estudiante por nombre con FirstOrDefault; si no se encuentra, se informa el error en lugar de intentar eliminar un elemento inexistente.

Módulo 2: Cola de Atención al Cliente — FIFO (Queue<string>)

Este módulo utiliza una *Queue<string>* para modelar una fila de atención al cliente, donde *Enqueue()* registra la llegada de un nuevo cliente al final de la fila, y *Dequeue()* atiende y retira al cliente que lleva más tiempo esperando, es decir, al primero de la fila.

```
Queue<string> colaClientes = new Queue<string>();

public void RegistrarLlegadaCliente(string nombreCliente)
{
    if (colaClientes.Contains(nombreCliente))
    {
        Console.WriteLine($"[Error]: El cliente '{nombreCliente}' ya se encuentra en
la fila de espera.");
        return;
    }
    colaClientes.Enqueue(nombreCliente);
    Console.WriteLine($"[Éxito]: Cliente '{nombreCliente}' añadido a la fila de
espera.");
}

public void AtenderSiguienteCliente()
{
    if (colaClientes.Count == 0)
    {
        Console.WriteLine("[Error]: No hay clientes en espera para atender.");
        return;
    }
    string clienteAtendido = colaClientes.Dequeue();
    Console.WriteLine($"[Atención en Curso]: Atendiendo y retirando de la fila al
cliente: '{clienteAtendido}'.");
    Console.WriteLine($"[Estado]: Quedan {colaClientes.Count} cliente(s) en espera.");
}
```

Antes de registrar la llegada de un cliente, se valida con Contains() que ese cliente no esté ya en la fila, evitando duplicados. Antes de atender, se valida que la cola no esté vacía (Count == 0), evitando una excepción al invocar Dequeue() sobre una colección sin elementos. El orden de atención respeta estrictamente el orden de llegada, que es la esencia del comportamiento FIFO.

Módulo 3: Pila de Tareas Pendientes — LIFO (Stack<string>)

Este módulo utiliza una *Stack<string>* para controlar tareas pendientes, donde *Push()* apila una nueva tarea en el tope, y *Pop()* marca como completada y retira precisamente la tarea más reciente, es decir, la que se encuentra en el tope de la pila.

```
Stack<string> pilaTareas = new Stack<string>();

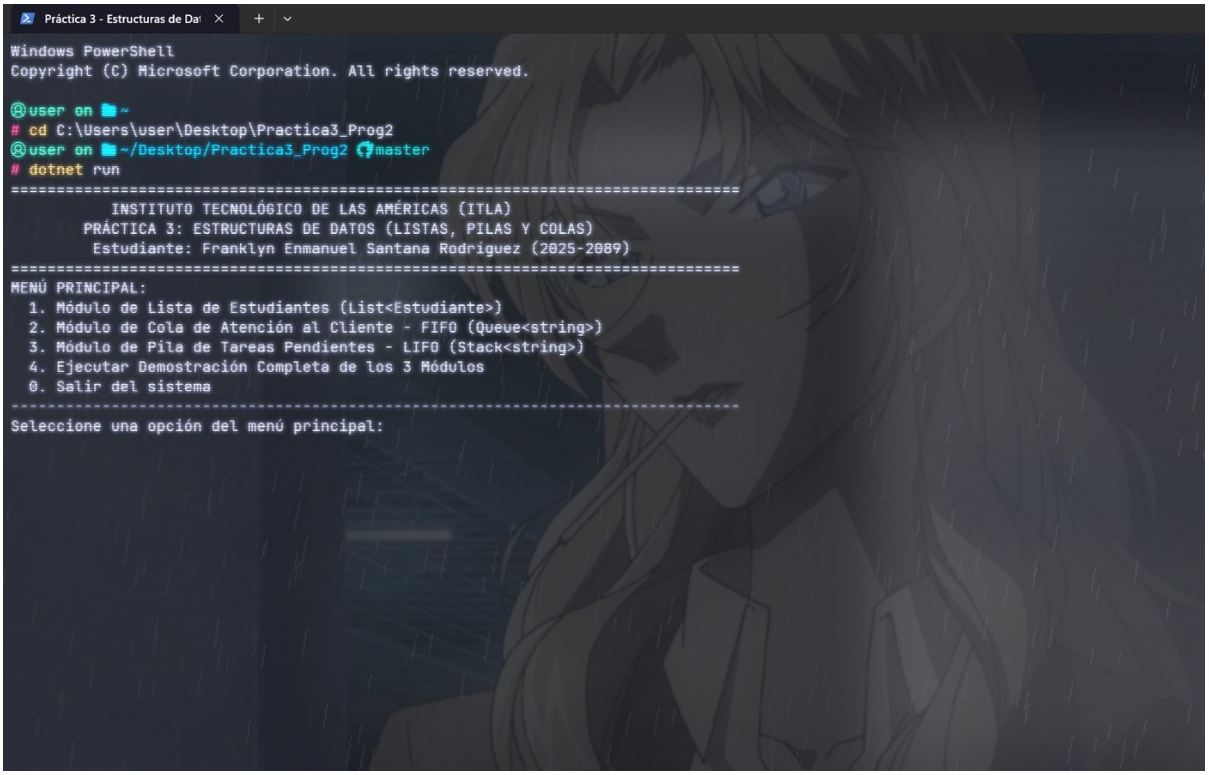
public void AgregarTareaPendiente(string descripcionTarea)
{
    pilaTareas.Push(descripcionTarea);
    Console.WriteLine($"[Éxito]: Tarea '{descripcionTarea}' añadida al tope de la
pila.");
}

public void CompletarTareaSuperior()
{
    if (pilaTareas.Count == 0)
    {
        Console.WriteLine("[Error]: No hay tareas pendientes en la pila.");
        return;
    }
    string tareaCompletada = pilaTareas.Pop();
    Console.WriteLine($"[Tarea Completada]: Se completó y eliminó de la pila la tarea:
'{tareaCompletada}'.");
    Console.WriteLine($"[Estado]: Quedan {pilaTareas.Count} tarea(s) pendiente(s) en
la pila.");
}
```

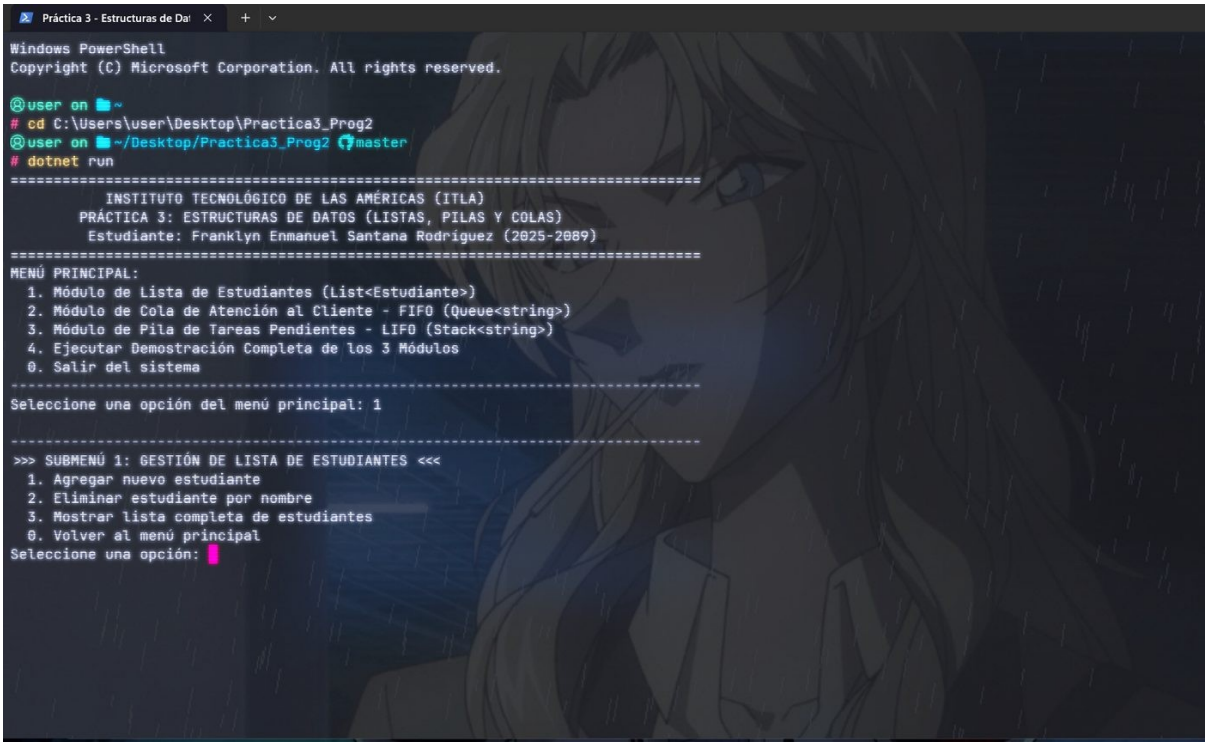
De forma análoga a los módulos anteriores, antes de invocar Pop() se valida que la pila no esté vacía (Count == 0), evitando una excepción en tiempo de ejecución. A diferencia de la cola, aquí siempre se retira la última tarea agregada, no la más antigua, lo cual es exactamente el comportamiento LIFO esperado: la tarea más reciente es la primera en resolverse.

Evidencias de Ejecución

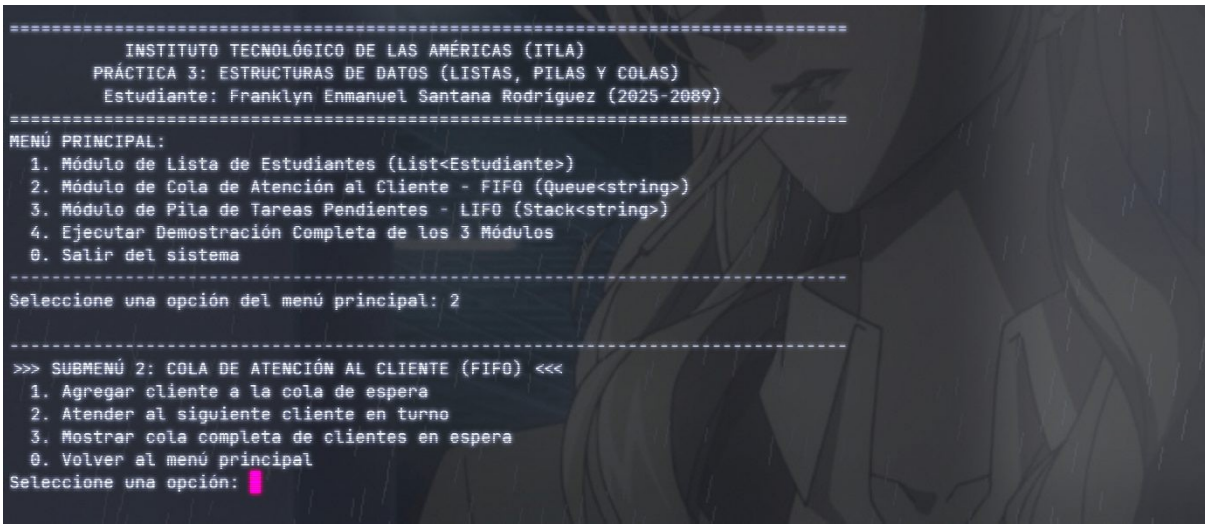
Las siguientes capturas muestran la ejecución del proyecto en la terminal mediante *dotnet run*, recorriendo el menú principal, cada uno de los tres submenús, y la demostración completa de los tres módulos con sus respectivas validaciones de negocio.



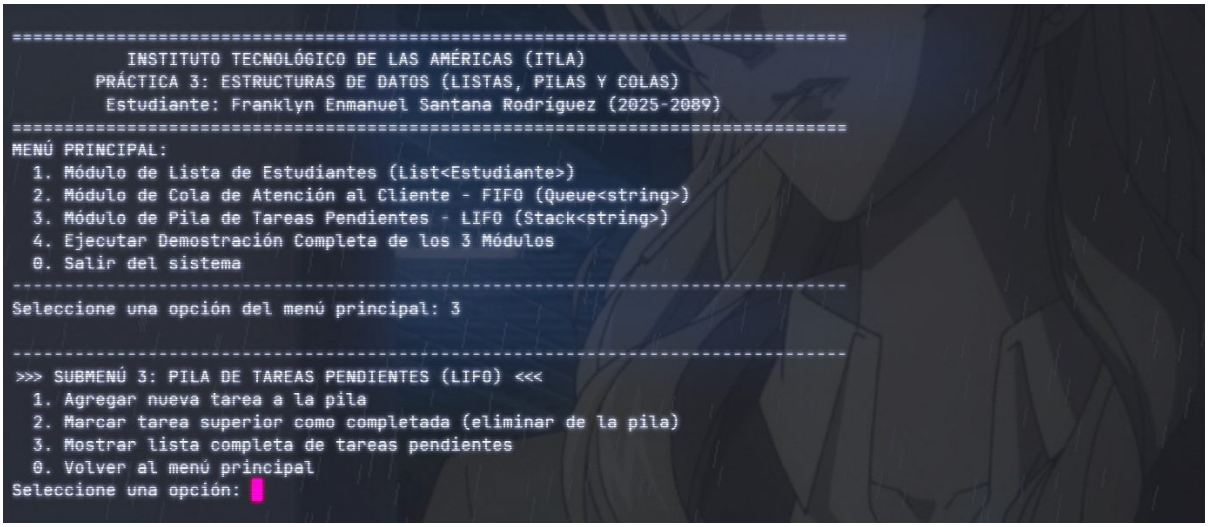
Captura 1 — Menú principal del sistema con las cuatro opciones de módulos y la opción de salida.



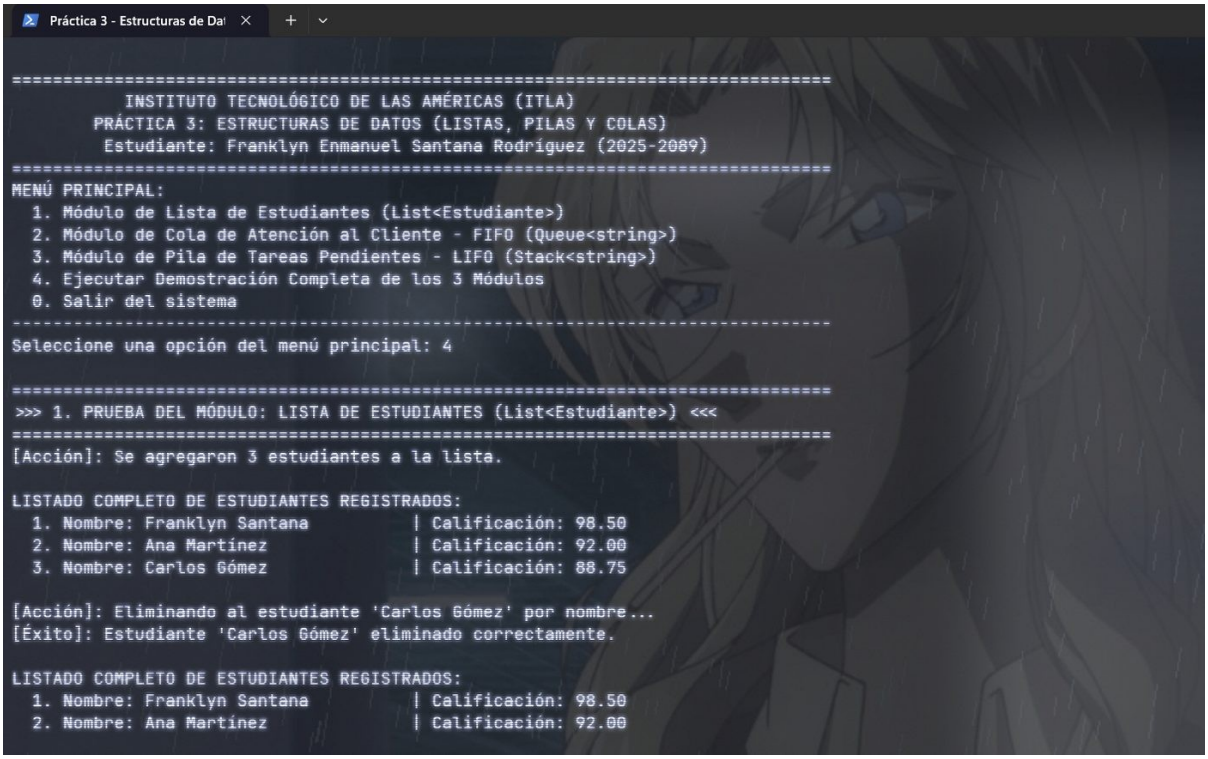
Captura 2 — Submenú 1: Gestión de la Lista de Estudiantes (List<Estudiante>).



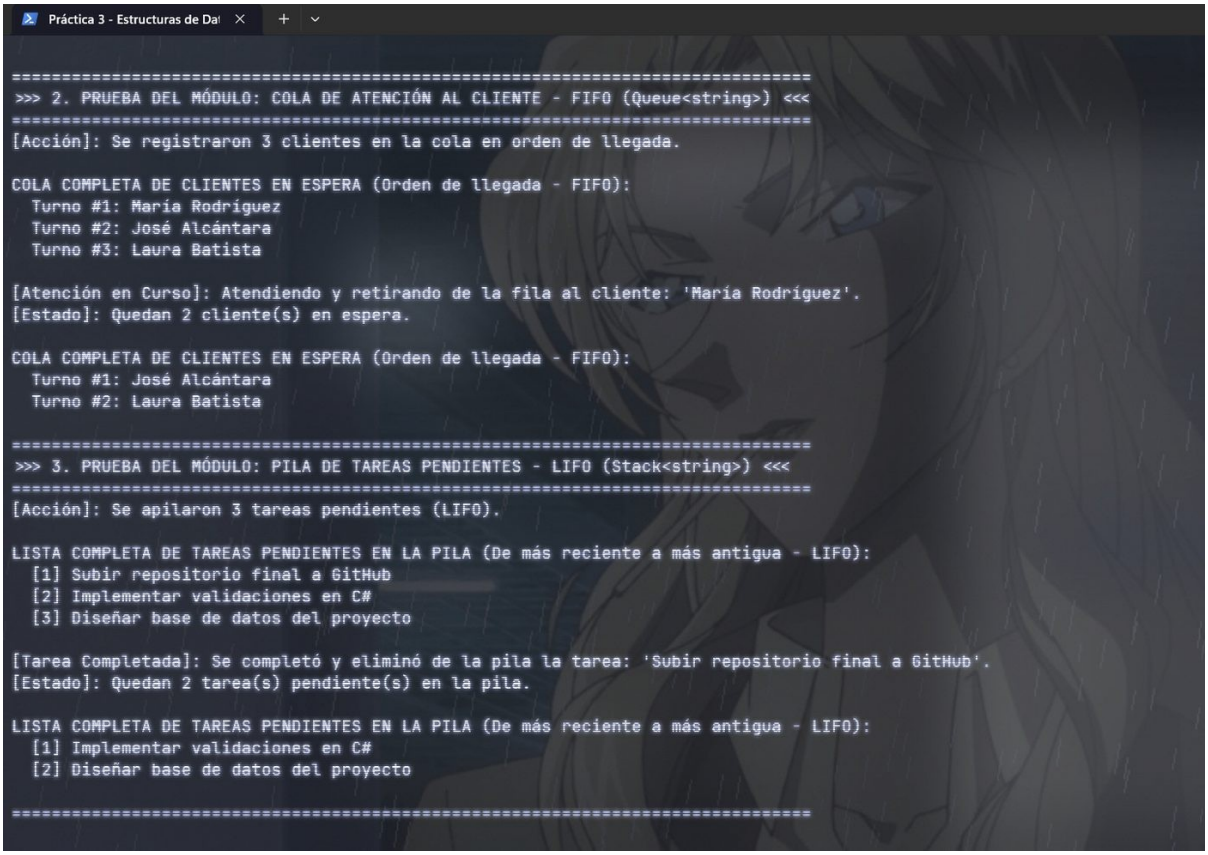
Captura 3 — Submenú 2: Cola de Atención al Cliente — FIFO (Queue<string>).



Captura 4 — Submenú 3: Pila de Tareas Pendientes — LIFO (Stack<string>).



Captura 5 — Demostración completa: prueba del módulo de Lista de Estudiantes, incluyendo la inserción, listado y eliminación por nombre.



Captura 6 — Demostración completa: prueba de la Cola de Atención al Cliente (FIFO) y de la Pila de Tareas Pendientes (LIFO).

En la Captura 5 se observa cómo, al agregar tres estudiantes (Franklyn Santana, Ana Martínez y Carlos Gómez) y luego eliminar a Carlos Gómez por nombre, la lista resultante conserva únicamente a los dos estudiantes restantes, confirmando el correcto funcionamiento de List<Estudiante>. En la Captura 6, la cola de clientes atiende primero a María Rodríguez por ser la primera en llegar (comportamiento FIFO), mientras que la pila de tareas completa primero “Subir repositorio final a GitHub”, por ser la última tarea agregada al tope (comportamiento LIFO), demostrando con claridad la diferencia de comportamiento entre ambas estructuras.

Conclusión

El desarrollo de esta práctica demuestra que la elección adecuada entre una Lista, una Cola o una Pila no es una decisión arbitraria, sino una decisión de diseño que impacta directamente tanto el rendimiento como la fidelidad con la que el sistema modela un flujo de trabajo del mundo real. Una List<T> resulta ideal cuando se necesita acceso indexado y búsquedas flexibles sobre toda la colección, como en el registro de estudiantes; una Queue<T> modela con precisión cualquier proceso de atención por orden de llegada, como una fila de clientes, gracias a su comportamiento FIFO; y una Stack<T> resulta la estructura natural para el control de tareas pendientes o flujos donde lo más reciente debe resolverse primero, gracias a su comportamiento LIFO. Dominar estas tres estructuras —y, sobre todo, reconocer en qué escenario aplica cada una— es una habilidad fundamental para escribir software eficiente, correcto y alineado con la lógica real del problema que se está resolviendo.